

学号： 2104230414



沈阳建筑大学

智能手机应用程序开发 综合设计报告

2023 年 秋 季学期

设计题目： 简约记账 APP

学 院： 计算机科学与工程学院

专业班级： 计算机 2101 班

姓 名： 张清晨

一、题目名称：简约记账 APP 系统设计与实现。

二、系统分析与设计

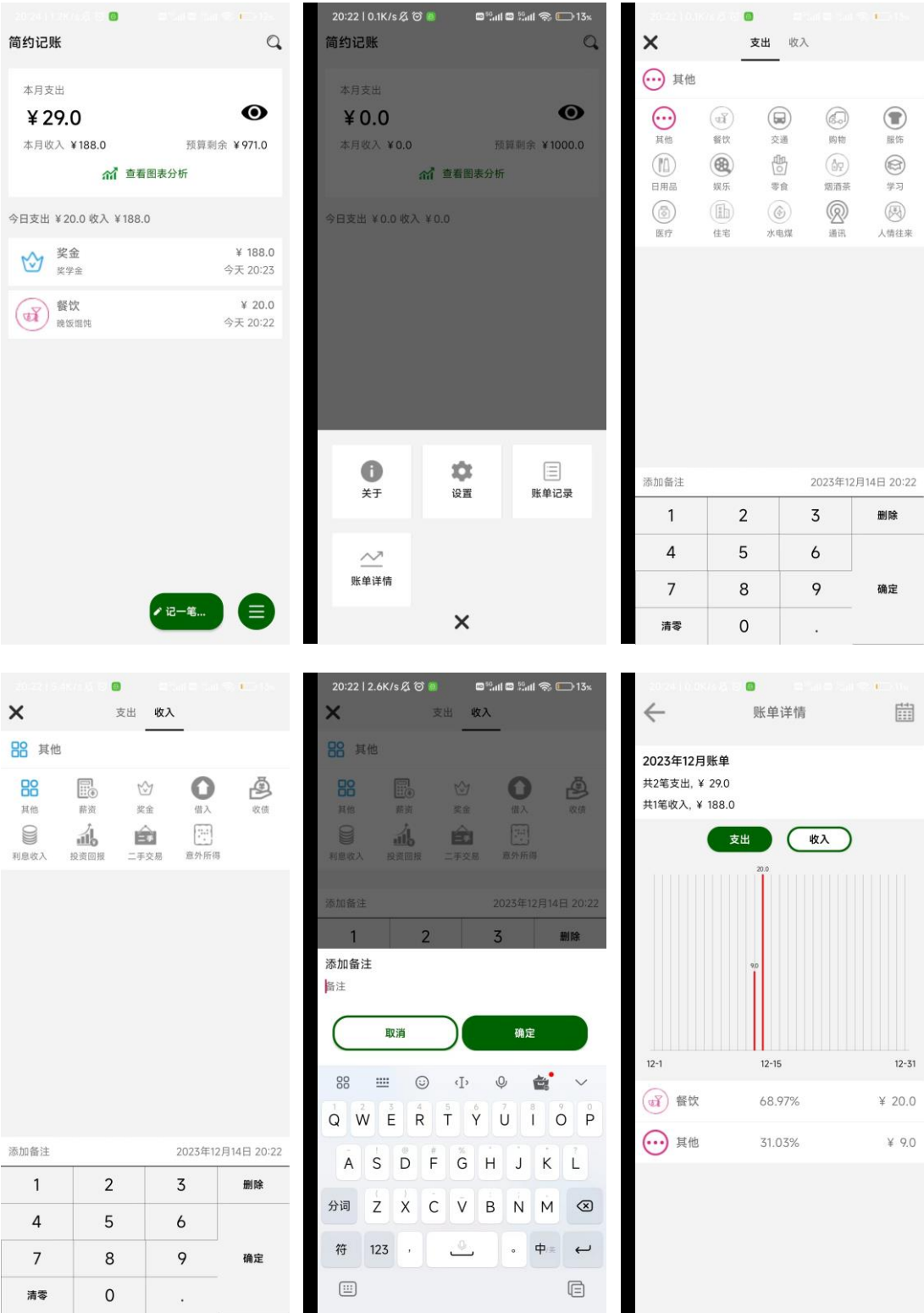
1、需求分析。

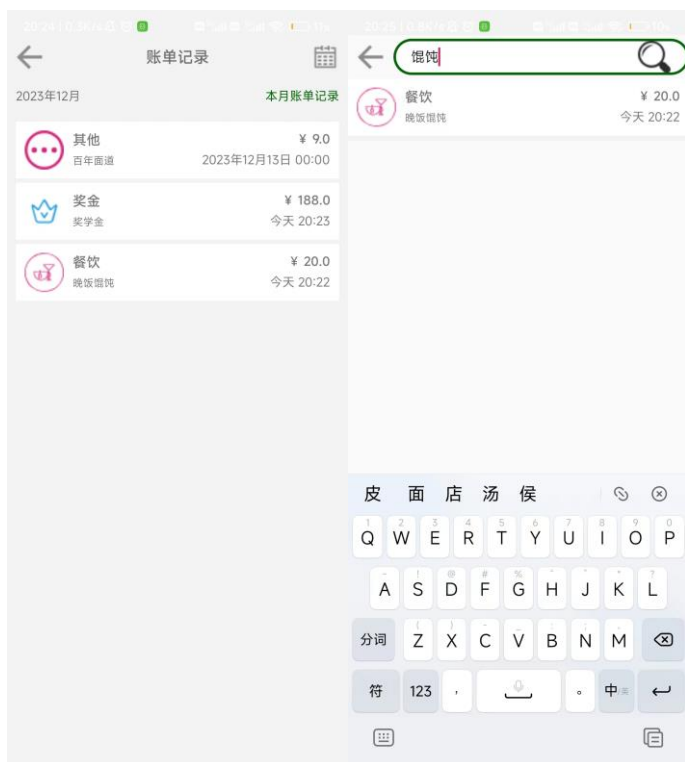
随着生活水平的提高，人们对于个人财务管理的需求也越来越强烈。然而，市面上的记账软件往往功能繁杂，操作复杂，不符合简单快捷的理念。因此，我们开发了一个基于安卓平台的简约记账应用，旨在为用户提供一个轻便易用的个人记账工具。

2、数据分析与设计。

- **数据库选择：**我选择使用 SQLite 数据库，SQLite 是一种嵌入式关系型数据库管理系（RDBMS），它提供了一个轻量级、零配置的数据库引擎。
- **数据可视化工具：**图表库 MPAndroidChart 是一个强大而灵活的开源图表库，用于在 Android 应用程序中显示各种类型的图表和图形。它提供了丰富的图表类型，包括折线图、柱状图、饼图、散点图、雷达图等，以及支持各种自定义和交互功能。
- **数据库创建的表：**
 - 收入类型表（typeNameIncome）： 存储收入账单的类型
[id, typename, kind]
 - 支出类型表(typeExpenditure)： 存储支出账单的类型
[id, typename, kind]
 - 收入账单记录(accountIncome)： 存储收入账单的具体记录
[id, typename, beizhu, money, time, year, month, day, kind]
 - 支出账单记录(accountExpenditure)： 存储支出账单的具体记录
[id, typename, beizhu, money, time, year, month, day, kind]
 - 历史记录表(history)： 存储所有账单的历史记录
[id, typename, beizhu, money, time, year, month, day, kind]

3、界面设计。

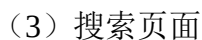
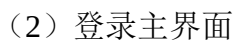


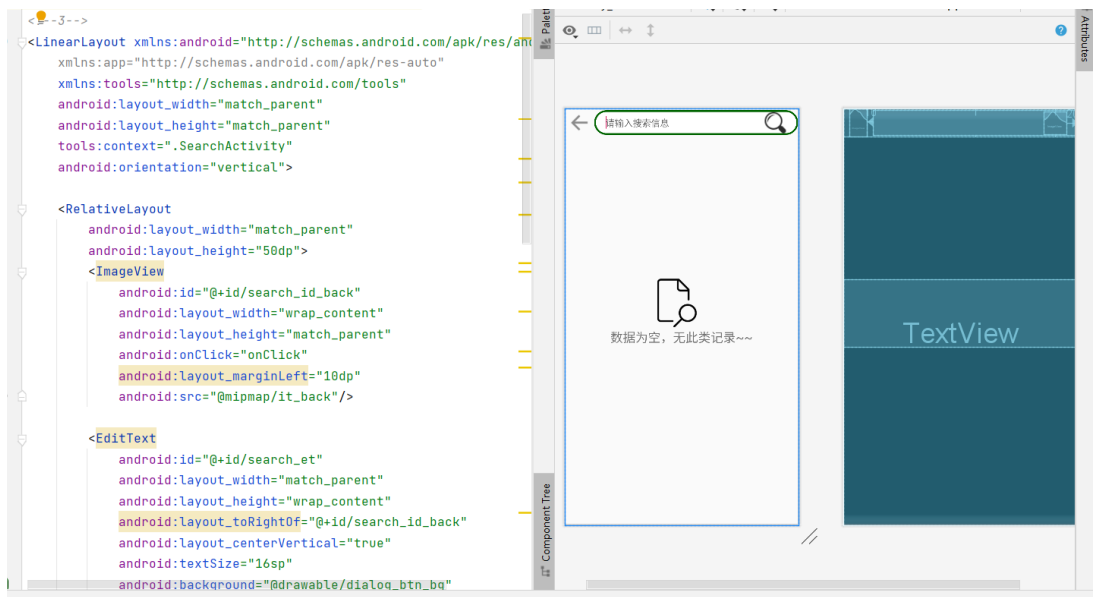


4、功能设计。

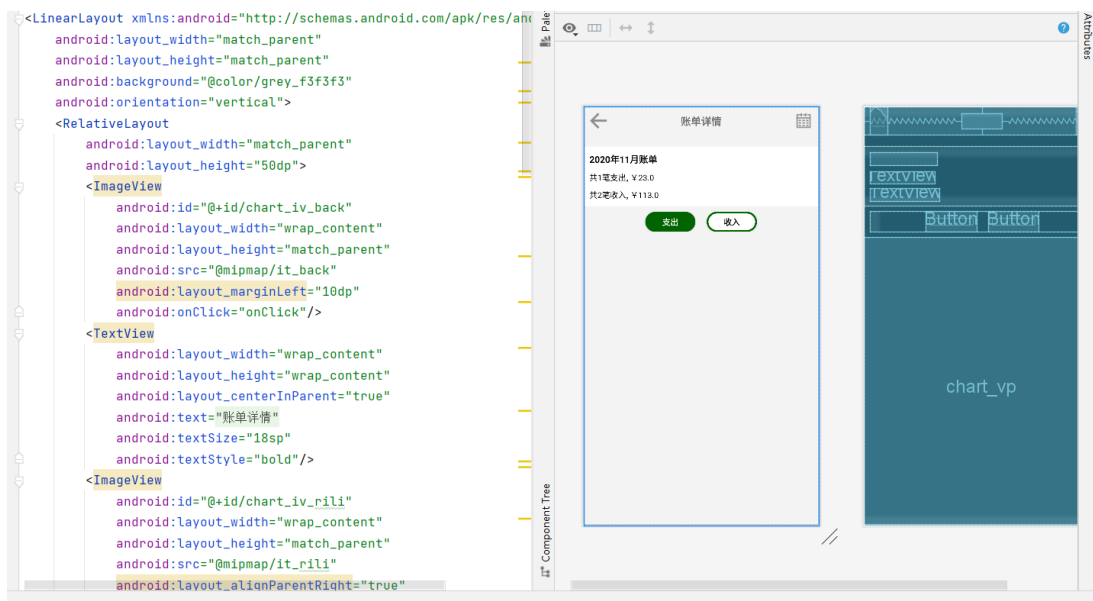
- **简易记账：**记录当天收支情况，包括收入和支出的金额、类别、备注等信息。
- **账单分析：**计算当天以及当月共收支总钱数，显示在主界面上，方便用户了解自己的财务状况；对于每月收支情况进行图表分析，展示用户的收支比例、趋势、变化等数据。
- **历史账单：**查看历史账单，按照日期或者类别进行筛选，查看每一笔收支的详细信息。
- **搜索：**搜索用户关于某种记录的全部收支情况，例如输入“餐饮”可以查看所有与餐饮相关的收支记录。

(1) 历史账单记录界面

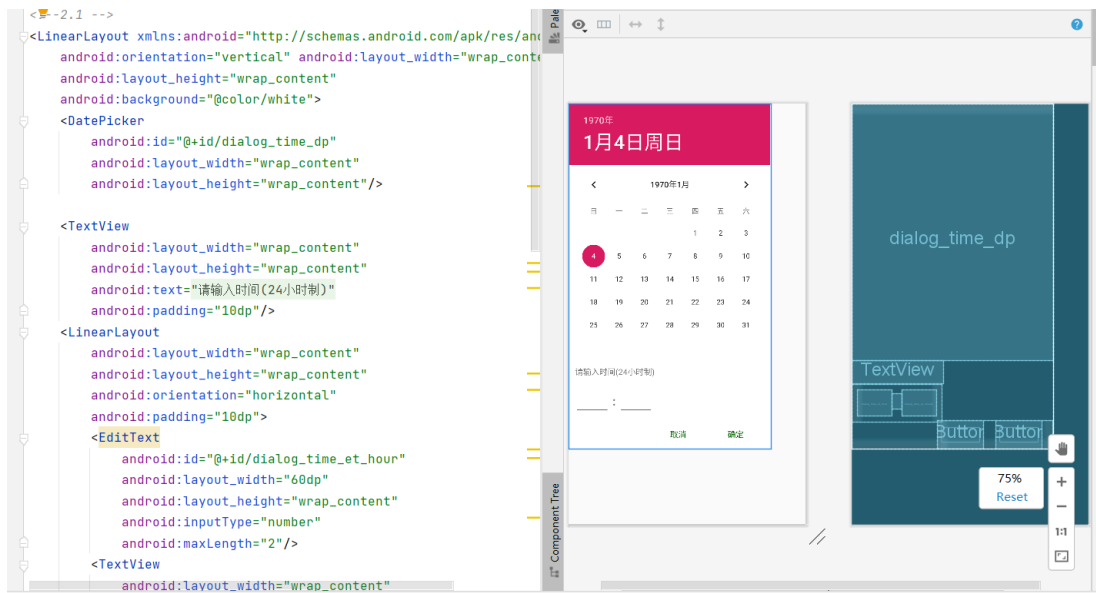




(4) 账单详细界面



(5) 时间选择界面



2、程序实现。

开发工具：Android Studio 是一个由谷歌开发的集成开发环境（IDE），用于开发 Android 应用程序。它是专门为 Android 平台开发而设计的，并且提供了丰富的工具和功能，使开发人员能够创建高质量的应用程序。

开发语言：Java 是一种广泛使用的编程语言，最初由 Sun Microsystems（后被 Oracle 收购）于 1995 年推出。它是一种面向对象的语言，被设计用于创建跨平台的应用程序。

（1）获取当前时间，在每次进行收入或支出的时候，刚进去时候自动填入当前时间。

```
private void setInitTime() {
    Date date = new Date();
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy 年 MM 月 dd 日 HH:mm");
    String time = sdf.format(date);
    timeTv.setText(time);
    accountBean.setTime(time);

    Calendar calendar = Calendar.getInstance();
    int year = calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH)+1;
    int day = calendar.get(Calendar.DAY_OF_MONTH);
    accountBean.setYear(year);
}
```

```

        accountBean.setMonth(month);
        accountBean.setDay(day);
    }

```

(2) 显示日历对话框，在进行账单记录查看时，会弹出一个日历进行时间的选择。

```

private void showCalendarDialog() {
    CalendarDialog dialog = new CalendarDialog(this, selectPos,
selectMonth);
    dialog.show();
    dialog.setDialogSize();
    dialog.setOnRefreshListener(new
CalendarDialog.OnRefreshListener() {
        @Override
        public void onRefresh(int selPos, int year, int month) {
            MonthChartActivity.this.selectPos = selPos;
            MonthChartActivity.this.selectMonth = month;
            initStatistics(year, month);
            incomChartFragment.setDate(year, month);
            outcomChartFragment.setDate(year, month);
        }
    });
}

```

(3) 记载页的适配器，将 Fragment 绑定到 ViewPager 中，并处理其生命周期管理和界面切换

```

public class RecordPagerAdapter extends FragmentPagerAdapter {
    List<Fragment>fragmentList;
    String[]titles = {"支出", "收入"};
    public RecordPagerAdapter(@NonNull FragmentManager fm,
List<Fragment>fragmentList) {
        super(fm);
        this.fragmentList = fragmentList;
    }
    @NonNull
    @Override
    public Fragment getItem(int position) {
        return fragmentList.get(position);
    }
}

```

```

    }

    @Override
    public int getCount() {
        return fragmentList.size();
    }

    @Nullable
    @Override
    public CharSequence getPageTitle(int position) {
        return titles[position];
    }
}

```

(4) 负责管理数据库的类，主要对于表当中的内容进行操作，增删改查
初始化数据库对象

```

public static void initDB(Context context) {
    DBOpenHelper helper = new DBOpenHelper(context); //得到帮助类对象
    db = helper.getWritableDatabase(); //得到数据库对象
}

```

读取数据库当中的数据，写入内存集合里， kind :表示收入或者支出

```

public static List<TypeBean> getTypeList(int kind) {
    List<TypeBean> list = new ArrayList<>();
    //读取 typetb 表当中的数据
    String sql = "select * from typetb where kind = " + kind;
    Cursor cursor = db.rawQuery(sql, null);
    // 循环读取游标内容，存储到对象当中
    while (cursor.moveToNext()) {
        String typename =
        cursor.getString(cursor.getColumnIndexOrThrow("typename"));
        int imageId =
        cursor.getInt(cursor.getColumnIndexOrThrow("imageId"));
        int sImageId =
        cursor.getInt(cursor.getColumnIndexOrThrow("sImageId"));
        int kind1 =

```

```

cursor.getInt(cursor.getColumnIndexOrThrow("kind"));
        int id =
cursor.getInt(cursor.getColumnIndexOrThrow("id"));
        TypeBean typeBean = new TypeBean(id, typename, imageId,
sImageId, kind);
        list.add(typeBean);
    }
    return list;
}

```

(5) 获取记账表中某一天所有支出或者收入情况

数据库编写

```

public static List<AccountBean>getAccountListOneDayFromAccounttb(int
year,int month,int day){
    List<AccountBean>list = new ArrayList<>();
    String sql = "select * from accounttb where year=? and month=? and
day=? order by id desc";
    Cursor cursor = db.rawQuery(sql, new String[] {year + "", month + "",
day + ""});
    //遍历符合要求的每一行数据
    while (cursor.moveToNext()) {
        int id = cursor.getInt(cursor.getColumnIndexOrThrow("id"));
        String typename =
cursor.getString(cursor.getColumnIndexOrThrow("typename"));
        String beizhu =
cursor.getString(cursor.getColumnIndexOrThrow("beizhu"));
        String time =
cursor.getString(cursor.getColumnIndexOrThrow("time"));
        int sImageId =
cursor.getInt(cursor.getColumnIndexOrThrow("sImageId"));
        int kind =
cursor.getInt(cursor.getColumnIndexOrThrow("kind"));
        float money =
cursor.getFloat(cursor.getColumnIndexOrThrow("money"));
        AccountBean accountBean = new AccountBean(id, typename,
sImageId, beizhu, money, time, year, month, day, kind);
        list.add(accountBean);
    }
}

```

```
return list;
}
```

数据库编写，获取记账表当中某一月的所有支出或者收入情况

```
public static
List<AccountBean>getAccountListOneMonthFromAccounttb(int year,int
month) {
    List<AccountBean>list = new ArrayList<>();
    String sql = "select * from accounttb where year=? and month=? order
by id desc";
    Cursor cursor = db.rawQuery(sql, new String[]{year + "", month +
""});
    //遍历符合要求的每一行数据
    while (cursor.moveToNext()) {
        int id = cursor.getInt(cursor.getColumnIndexOrThrow("id"));
        String typename =
cursor.getString(cursor.getColumnIndexOrThrow("typename"));
        String beizhu =
cursor.getString(cursor.getColumnIndexOrThrow("beizhu"));
        String time =
cursor.getString(cursor.getColumnIndexOrThrow("time"));
        int sImageId =
cursor.getInt(cursor.getColumnIndexOrThrow("sImageId"));
        int kind =
cursor.getInt(cursor.getColumnIndexOrThrow("kind"));
        float money =
cursor.getFloat(cursor.getColumnIndexOrThrow("money"));
        int day = cursor.getInt(cursor.getColumnIndexOrThrow("day"));
        AccountBean accountBean = new AccountBean(id, typename,
sImageId, beizhu, money, time, year, month, day, kind);
        list.add(accountBean);
    }
    return list;
}
```

获取某一天的支出或者收入的总金额 kind-> 支出：0 收入：1

```
public static float getSumMoneyOneDay(int year, int month, int day, int
kind) {
    float total = 0.0f;
```

```

String sql = "select sum(money) from accounttb where year=? and
month=? and day=? and kind=?";
Cursor cursor = db.rawQuery(sql, new String[] {year + "", month + "",
day + "", kind + ""});
//遍历
if (cursor.moveToFirst()) {
    float money =
cursor.getFloat(cursor.getColumnIndexOrThrow("sum(money)"));
    total = money;
}
return total;
}

```

统计某一月的支出或者收入的总金额 收入-1 支出-0

```

public static int getCountItemOneMonth(int year, int month, int kind) {
    int total=0;
    String sql="select count(money) from accounttb where year=? and
month=? and kind=?";
    Cursor cursor=db.rawQuery(sql, new
String[] {year+ "", month+ "", kind+ ""});
    if(cursor.moveToFirst()) {
        int
count=cursor.getInt(cursor.getColumnIndexOrThrow("count(money)"));
        total=count;
    }
    return total;
}

```

删除 accounttb 表格当中所有数据

```

public static void deleteAllAccount() {
    String sql = "delete from accounttb";
    db.execSQL(sql);
}

```

获取这个月当中某一天收入支出最大的金额，金额是多少

```

public static float getMaxMoneyOneDayInMonth(int year, int month, int
kind) {
    String sql = "select sum(money) from accounttb where year=? and
month=? and kind=? group by day order by sum(money) desc";
    Cursor cursor = db.rawQuery(sql, new

```

```
String[] {year+"", month+"", kind+""});
    if (cursor.moveToFirst()) {
        float money =
cursor.getFloat(cursor.getColumnIndexOrThrow("sum(money)"));
        return money;
    }
    return 0;
}
```

3、运行与测试。

(1) 隐藏收入、支出、预算信息



(2) 给收入或支出信息添加备注信息



添加备注
早点



(3) 设置当月的预算金额

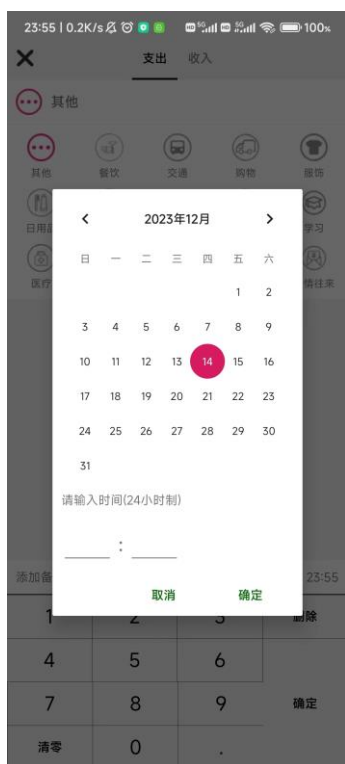


设置预算

¥ 2000



（4）给账单信息设置时间



4、难点和亮点。

难点：

操作 SQLite 数据库时，由于账单所列的数据非常详细，有类型，年份，月份，收入和支出之分，所以数据库的书写难度加大。

账单分析页面图表的插入，账单分析页面，由于需要读取数据库，来生成图表，此类在网上引用的时候，出现了引用错误，还生成许多 bug。

亮点：

1. 简洁直观的用户界面：

设计简约而直观的用户界面，使用户能够迅速了解主要功能，轻松上手。避免繁琐的操作，简单几步即可完成记账。

2. 个性化记账分类：

允许用户自定义记账分类，满足不同用户的个性化需求。这样用户可以更好地适应自己的财务管理习惯。

3. 实时财务概览：

提供实时的总体财务概览，包括总收入、总支出、净收入等关键指标，让用户一目了然地了解自己的财务状况。

4. 图表统计和趋势分析：

通过图表和趋势分析，直观展示用户的支出和收入情况，帮助用户更好地理解

自己的财务状况，发现潜在的消费趋势。

5. 轻量化和高性能：

保持应用轻量化，减少资源占用，提高运行速度，确保用户在使用过程中的顺畅体验。

四、总结体会

通过移动应用开发这门课的综合设计，我了解了软件开发的基本流程方法，也学会了去使用 Android studio 这款软件，同时也认识到了自己的不足，在在这门课程上还需要多下功夫；也学习了 SQLite 轻量数据库的使用，在设计和实现简约记账 APP 的过程中，注重提供用户友好的界面、智能的记账体验以及全面的财务管理功能。

在总体设计中，我不仅注重了技术的创新和功能的完备，更关注了用户体验和用户需求。通过这样的设计理念，我们相信这款简约记账 APP 能够为用户提供更加便捷、智能、个性化的财务管理体验。我也学习了一个软件开发的过程中，设计是很重要的一个步骤，设计决定了软件的成败与否。